
COMP206 Learning Journal

Hector Barquero

Jul 18, 2026

CONTENTS:

1	Assignment three journal	1
1.1	Program design	2
1.2	Overview and reflections	4
1.3	Readings	5
1.4	Practice problems	10
1.5	Sources and references	14

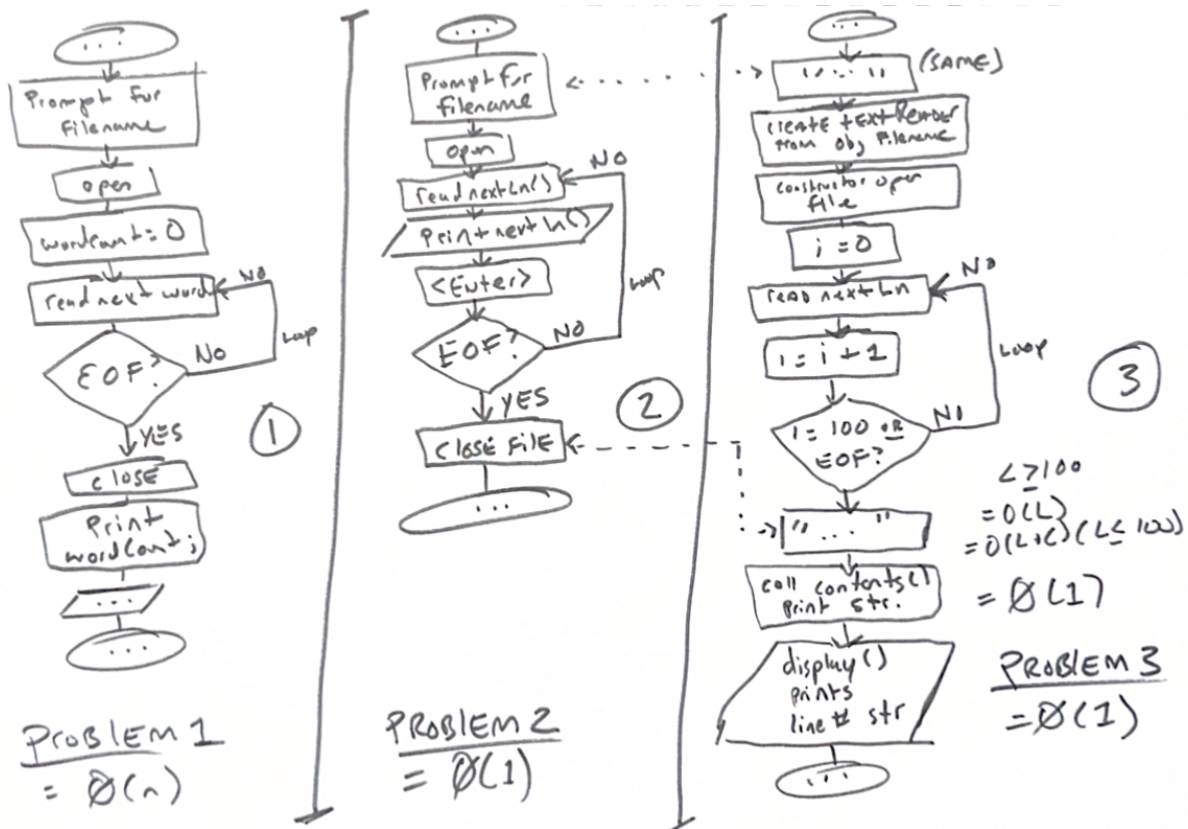
ASSIGNMENT THREE JOURNAL

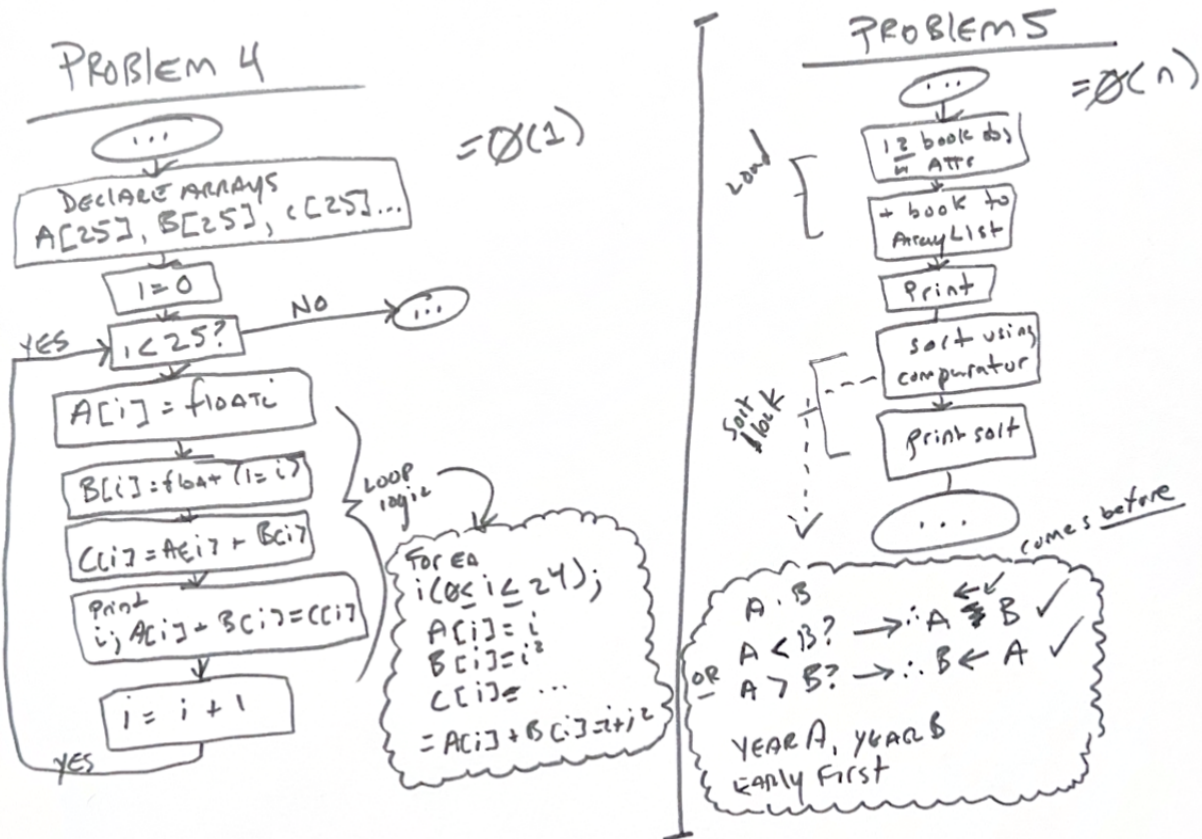
Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

 Tip**How to use this journal**

Learning journal contains program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter.

1.1 Program design





1.1.1 Assignment 3, Problem 1

- Added command-line filename handling and documented that choice. (v1.0)
- Opened the requested file with `ifstream` and added an open-failure check. (v1.0-v1.1)
- Counted words with `while (inputFile >> word)` so all whitespace types act as delimiters. (v1.1)
- Added usage output for missing or extra command-line arguments. (v1.1)
- Tested the program with `excerpt.txt`, an empty file and mixed spaces, tabs and line breaks. (v1.2)
- Verified that the supplied `excerpt.txt` contains 327 whitespace-separated words. (v1.2)

1.1.2 Assignment 3, Problem 2

- Added a filename prompt using `getline` so filenames may contain spaces. (v1.0)
- Read the file one complete line at a time with `getline(inputFile, line)`. (v1.0)
- Added a pause after every displayed line that waits for the Enter key. (v1.0-v1.1)
- Added one-based line numbers to make each displayed line easier to verify. (v1.1)
- Added checks for a blank filename and a file that cannot be opened. (v1.1)
- Tested normal text lines, blank lines and the supplied `excerpt.txt`. (v1.2)

1.1.3 Assignment 3, Problem 3

- Added a `TextFileReader` class containing an array of exactly 100 strings. (v1.0)
- Added a default constructor and a filename constructor that reads at most 100 lines. (v1.0)
- Added `contents()` to combine stored lines into one newline-separated string. (v1.1)
- Added `display()` to print each stored line using one-based line numbers. (v1.1)
- Added `isOpen()` and `size()` to support file-error checks and test verification. (v1.1-v1.2)
- Added `TextFileReaderDemo::run()` and forwarded the required global C++ `main()` to it. (v1.2)
- Tested the default constructor, missing files, short files and the 100-line storage limit. (v1.3)

1.1.4 Assignment 3, Problem 4

- Added three `array<double, 25>` objects for counter values, squares and sums. (v1.0)
- Filled the first array with counter values from 0 through 24. (v1.0)
- Filled the second array using `value * value` for each corresponding element. (v1.0)
- Filled the third array by adding corresponding elements from the first two arrays. (v1.0)
- Added formatted output using `counter; element + element = element`. (v1.1)
- Verified the first, middle and final rows and confirmed that exactly 25 rows are displayed. (v1.1)

1.1.5 Assignment 3, Problem 5

- Reused the private `Book` attributes and public `get/set` methods from Assignment 2. (v1.0)
- Added a `Bookshelf` class containing a vector of twelve distinct `Book` objects. (v1.0)
- Displayed all twelve books in their original insertion order. (v1.1)
- Added `BookComparator::compareTo()` to compare titles first and publication years second. (v1.1-v1.2)
- Connected the comparator to `std::sort` through `operator()`. (v1.2)
- Added two books with the same title to verify that the earlier publication year sorts first. (v1.3)
- Verified the unsorted and sorted lists both contain all twelve books and all stored attributes. (v1.4)

1.2 Overview and reflections

1.2.1 Unit 7: Points to ponder

Hector Barquero posted Mar 21, 2026 8:35 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/350740/View>

I compared differences to javascript and the main differences I found were mutability. I forgot that there was a subtle difference, you can't mutate strings in js, but you can in cpp.

I also forgot that `s.find` is a member function in cpp and returns `string::npos` when there's nothing found, but in javascript it returns -1 (`indexOf`) and is a method you use on the string itself.

1.2.2 Unit 8: Points to ponder

Hector Barquero posted Jun 8, 2026 3:42 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/391920/View>

Reading this chapter took me a long time, since I took a small hiatus from AU coursework– but I’m back and was pleasantly surprised that things were ramping up in this chapter since we’ve moved from thinking about “just writing code” to now thinking about the tech debt of our mathematical calculations.

Instead of passing regular vars, we’re now looking at structs and compound types, doing things like call by values and call by refs.

This is fun for me because compared to javascript, c++ makes these concepts explicit. In js, objects are passed around and object shapes change at runtime whereas c++ forces you to think about the structure of your data, if you want to copy it, and how you’ll pass it within functions. It feels like a step away for me from writing code to now writing calculations, and that’s fun.

1.2.3 Unit 9: Points to ponder

Hector Barquero posted Jun 12, 2026 4:05 AM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/394111/View>

I found this chapter a lighter read since it was less about syntax and more about design choices. One neat difference from what I’m used to is that c++ lets you declare const with a reference, for example:

```
const &foo n
```

where you’re essentially saying: “hey use this original object, don’t copy it, and make it immutable”. Javascript doesn’t have that, so it’s neat for me to remember that’s a thing. Objects in js are mutable and passed around with reference-like behaviour by default, but in c++ you’re constantly thinking about choices between copying, modifying the original or a read only-like behaviour:

```
for ex: foo n <- copy foo &n <- modify original const n <- read only
```

This chapter was cool because it exposed design patterns and thinking less than syntax, and exposed what other languages like javascript are hiding from you.

1.3 Readings

1.3.1 Unit 7

ThinkCScpp 7.1: Containers for strings

- A string stores an ordered sequence of characters in one object.
- `std::string` manages its own storage and can grow or shrink as needed.

ThinkCScpp 7.2: string variables

- A string variable can be declared, initialized, reassigned, and passed to functions.
- String literals use double quotes.
- `std::string` belongs to the standard library.

ThinkCScpp 7.3: Extracting characters from a string

- The subscript operator `[]` accesses a character by its zero-based index.
- The first character is at index `0`.
- Accessing an invalid index produces undefined behaviour.

ThinkCScpp 7.4: Length

- The `length` or `size` function returns the number of characters in a string.
- The final valid index is one less than the string's length.

ThinkCScpp 7.5: Traversal

- String traversal visits characters one at a time, usually with a loop.
- An index-based loop can read or modify each character.
- A range-based loop is useful when the index isn't needed.
- Traversal normally runs in linear time relative to the string length.

ThinkCScpp 7.6: A run-time error

- A run-time error can occur when a program accesses a string outside its valid range.
- The code may compile even though the invalid index only appears during execution.
- Boundary tests help expose off-by-one errors.

ThinkCScpp 7.7: The find function

- `string::find` returns the first position where a character or substring appears.
- It returns `string::npos` when no match exists.
- Searches can optionally begin at a specified index.
- The result should be checked before it's used as an index.

ThinkCScpp 7.8: Our own version of find

- A custom find function scans characters until it locates the target.
- It should return either the matching position or a clear failure value.
- Writing the search manually demonstrates traversal, conditions, and early return.

ThinkCScpp 7.9: Looping and counting

- A counter can track how many characters satisfy a condition during traversal.
- The counter starts at zero and increases when a match is found.
- Counting requires examining every character unless an earlier result is sufficient.
- The operation is generally $O(n)$.

ThinkCScpp 7.10: Increment and decrement operators

- `++` adds one to a variable and `--` subtracts one.
- Prefix and postfix forms differ in the value produced by the expression.
- In simple loop updates, either form usually has the same practical effect.

ThinkCScpp 7.11: String concatenation

- The `+` operator joins strings into a new string.
- `+=` appends text to an existing string.
- At least one operand must be a `string` for many concatenation expressions.
- Repeated concatenation can require additional memory allocations.

ThinkCScpp 7.12: strings are mutable

- Individual string characters can be changed through valid indexes.
- Strings can also be expanded, shortened, inserted into, or erased.
- Mutation changes the existing string rather than creating a separate result.

ThinkCScpp 7.13: strings are comparable

- Comparison operators compare strings lexicographically.
- Character case affects comparison results.
- Equality requires the same characters in the same order.
- Lexicographic order isn't the same as comparing string lengths.

ThinkCScpp 7.14: Character classification

- Functions such as `isalpha`, `isdigit`, and `isspace` classify characters.
- `toupper` and `tolower` convert character case.
- These functions are provided by `<cctype>`.
- Character arguments should be converted to `unsigned char` when plain `char` might be negative.
- Classification functions return values usable as Boolean conditions.

ThinkCScpp 7.15: Other string functions

- Common string operations include `substr`, `insert`, `erase`, `replace`, and `append`.
- `substr` extracts part of a string using a starting position and optional length.
- Many functions return or modify positions measured from zero.

1.3.2 Unit 8**ThinkCScpp 8.1: Compound values**

- A compound value groups several related values into one object.
- Structures can combine members with different data types.
- Grouping related data makes function interfaces and program models clearer.

ThinkCScpp 8.2: Point objects

- A `Point` structure can represent coordinates using members such as `x` and `y`.
- Each object stores its own independent member values.

ThinkCScpp 8.3: Accessing instance variables

- The dot operator accesses a structure object's members.
- Member names are scoped by the structure type.
- A pointer to a structure uses the `->` operator instead.

ThinkCScpp 8.4: Operations on structures

- Structures can be copied, assigned, initialized, and passed to functions.
- Member values can participate in ordinary expressions.
- Comparison isn't automatically defined for every custom structure.
- Helper functions can implement operations specific to the structure.

ThinkCScpp 8.5: Structures as parameters

- Passing a structure to a function lets the function operate on the grouped data.
- The parameter type must match the structure type.
- Large structures are often passed by reference to avoid copying.

ThinkCScpp 8.6: Call by value

- Call by value gives the function a copy of the argument.
- Changes to the parameter don't affect the original object.
- Copying provides isolation but can add cost for large objects.

ThinkCScpp 8.7: Call by reference

- A reference parameter gives the function access to the caller's original object.
- Changes made through a non-const reference affect the caller.
- References avoid copying and can't be null after valid initialization.
- Use const references when the function should only read the object.

ThinkCScpp 8.8: Rectangles

- A rectangle can be represented using a corner point, width, and height.
- Nested structures let one compound object contain another.
- Representation choices affect how position and size calculations are written.

ThinkCScpp 8.9: Structures as return types

- A function can construct and return a complete structure object.
- Returning a structure is useful when one calculation produces several related values.
- Modern compilers usually optimize away unnecessary return copies.

ThinkCScpp 8.10: Passing other types by reference

- Reference parameters aren't limited to structures and can be used with built-in or library types.
- They can return multiple results through output parameters.
- `const` references allow efficient read-only access.
- Non-`const` references should clearly signal that an argument may change.

ThinkCScpp 8.11: Getting user input

- Input can be read directly into structure members.
- Each extraction should be checked before the values are trusted.
- Prompts should make the expected format clear.
- Invalid input can leave the stream in a failed state.
- `getline` may be needed when text fields contain spaces.

1.3.3 Unit 9**ThinkCScpp 9.1: Time**

- A `Time` structure can group hours, minutes, and seconds.
- The representation should define valid ranges for each member.
- Related values are easier to pass and return as one object.

ThinkCScpp 9.2: `printTime`

- A `printTime` function formats a `Time` object for output.
- Separating formatting from calculation keeps responsibilities clear.
- Padding may be used so minutes and seconds always occupy two digits.

ThinkCScpp 9.3: Functions for objects

- Functions can accept objects as parameters and perform type-specific operations.
- Related helper functions form an interface around the object's representation.
- The function name should describe the operation rather than the storage details.

ThinkCScpp 9.4: Pure functions

- A pure function returns a result without modifying its arguments or external state.
- The same inputs always produce the same output.
- Pure functions are easier to test and reuse.
- They reduce unexpected interactions between parts of a program.

ThinkCScpp 9.5: `const` parameters

- A `const` reference parameter prevents the function from modifying the supplied object.
- It avoids copying while documenting read-only intent.
- The compiler rejects attempts to change the parameter.

ThinkCScpp 9.6: Modifiers

- A modifier changes an existing object rather than returning a separate object.
- Modifier functions usually take non-const references.
- Their names should make the mutation clear.
- Callers must account for the original object's changed state.

ThinkCScpp 9.7: Fill-in functions

- A fill-in function writes a result into an object supplied by reference.
- It can avoid constructing a separate return value.
- Output parameters should be obvious from the function interface.

ThinkCScpp 9.8: Which is best?

- Pure functions are usually easier to reason about because they don't change their inputs.
- Modifiers can be useful when updating an existing object is the intended operation.
- Fill-in functions may help when several results must be returned.
- The best design depends on clarity, ownership, cost, and expected use.
- Consistency matters more than forcing one style everywhere.

ThinkCScpp 9.9: Incremental development versus planning

- Incremental development adds and tests one small behaviour at a time.
- Up-front planning defines the representation and major operations before implementation.
- Complex tasks often benefit from combining both approaches.
- Tests should be added as each step becomes functional.

ThinkCScpp 9.10: Generalization

- Generalization replaces fixed assumptions with parameters or reusable operations.
- A generalized function can handle more cases without duplicated code.
- Abstraction shouldn't make a simple solution harder to understand.

ThinkCScpp 9.11: Algorithms

- An algorithm is a finite, defined sequence of steps for solving a problem.
- Algorithms can be compared by correctness, clarity, time cost, and memory cost.
- The same problem may support several valid algorithms.
- Input size determines how performance scales.

1.4 Practice problems

1.4.1 Unit 7 practice problems

Find the Index of the First Occurrence in a String (Easy)

Return the first index where one string occurs inside another, or `-1` when it isn't found.

```

class Solution {
public:
    int strStr(string haystack, string needle) {
        if (needle.empty()) {
            return 0;
        }

        if (needle.size() > haystack.size()) {
            return -1;
        }

        for (size_t start = 0;
             start + needle.size() <= haystack.size();
             ++start) {

            bool matches = true;

            for (size_t offset = 0;
                 offset < needle.size();
                 ++offset) {

                if (haystack[start + offset] != needle[offset]) {
                    matches = false;
                    break;
                }
            }

            if (matches) {
                return static_cast<int>(start);
            }
        }

        return -1;
    }
};

```

1.4.2 Unit 8 practice problems

Rectangle Overlap (Easy)

Determine whether two rectangles share an area greater than zero.

```

class Solution {
private:
    struct Point {
        int x;
        int y;
    };

    struct Rectangle {
        Point bottomLeft;
        Point topRight;
    };

```

(continues on next page)

(continued from previous page)

```

Rectangle makeRectangle(const vector<int>& values) {
    return {
        {values[0], values[1]},
        {values[2], values[3]}
    };
}

bool overlaps(const Rectangle& first,
              const Rectangle& second) {
    bool separatedHorizontally =
        first.topRight.x <= second.bottomLeft.x ||
        second.topRight.x <= first.bottomLeft.x;

    bool separatedVertically =
        first.topRight.y <= second.bottomLeft.y ||
        second.topRight.y <= first.bottomLeft.y;

    return !separatedHorizontally &&
           !separatedVertically;
}

public:
    bool isRectangleOverlap(vector<int>& rec1,
                           vector<int>& rec2) {
        Rectangle first = makeRectangle(rec1);
        Rectangle second = makeRectangle(rec2);

        return overlaps(first, second);
    }
};

```

1.4.3 Unit 9 practice problems

Number of Days Between Two Dates (Easy)

Calculate the absolute number of days between two dates written as ``YYYY-MM-DD``.

```

class Solution {
private:
    struct Date {
        int year;
        int month;
        int day;
    };

    Date parseDate(const string& value) {
        return {
            stoi(value.substr(0, 4)),
            stoi(value.substr(5, 2)),
            stoi(value.substr(8, 2))
        };
    }
};

```

(continues on next page)

(continued from previous page)

```
}

bool isLeapYear(int year) {
    return year % 400 == 0 ||
           (year % 4 == 0 && year % 100 != 0);
}

int daysBeforeYear(int year) {
    int previousYear = year - 1;

    return previousYear * 365 +
           previousYear / 4 -
           previousYear / 100 +
           previousYear / 400;
}

int daysBeforeMonth(const Date& date) {
    const array<int, 12> monthLengths = {
        31, 28, 31, 30, 31, 30,
        31, 31, 30, 31, 30, 31
    };

    int total = 0;

    for (int month = 1; month < date.month; ++month) {
        total += monthLengths[month - 1];

        if (month == 2 && isLeapYear(date.year)) {
            ++total;
        }
    }

    return total;
}

int serialDay(const Date& date) {
    return daysBeforeYear(date.year) +
           daysBeforeMonth(date) +
           date.day;
}

public:
    int daysBetweenDates(string date1, string date2) {
        Date first = parseDate(date1);
        Date second = parseDate(date2);

        return abs(serialDay(first) - serialDay(second));
    }
};
```

1.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from <https://cppreference.com/cpp>
2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. <https://press.rebus.community/programmingfundamentals/>
3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. <https://rooksguide.org/>
4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. <https://www.greenteapress.com/thinkcpp/>
5. leetcode.com. (n.d.). *LeetCode*. Retrieved Jan 2025 - Aug 2026, from <https://leetcode.com/>